Guía Operativa de Desarrollo Agéntico y Harness Engineering

1. Fundamentos en Simple: ¿Cómo Funciona un LLM y Qué Son los Tokens?

Un **LLM (*Large Language Model* o Modelo Extenso de Lenguaje)** es el cerebro detrás de herramientas como ChatGPT, Claude o Gemini[cite: 7, 9]. Pero, a diferencia de una persona, un LLM **no "piensa" ni entiende la verdad**: es una calculadora gigante de probabilidades que predice qué viene después[cite: 7].

El ejemplo de tu perra con gorra (Aprendizaje por patrones)

Imagina que entrenas a una inteligencia artificial mostrándole **miles de fotos de tu perra** en distintas posiciones, luces y lugares. La IA no guarda una foto exacta como si fuera una fotocopiadora; lo que hace es aprender sus **patrones clave** (la forma de su hocico, el color del pelaje, la distancia entre sus ojos, sus orejas).

- Al día siguiente, le muestras una foto nueva de **tu perra usando una gorra**.
- Aunque nunca antes la haya visto con gorra, la IA detecta los rasgos que ya conoce, descarta lo que tapa la gorra y calcula: *"Hay un 98% de probabilidad de que sea tu perra"*.

Con el texto y el código hace exactamente lo mismo: tras leer millones de libros y programas, aprende los patrones del lenguaje y calcula qué palabra o línea de código tiene más probabilidad de seguir[cite: 7].

Tres conceptos clave para entenderlo:

1. **Tokens (Las piezas de Lego):** La IA no lee palabras enteras ni letras sueltas; corta el texto en fragmentos llamados *tokens* (por ejemplo, `"programación"` puede dividirse en `"program"` + `"ación"`)[cite: 7, 8]. Todo lo que lee y escribe se mide en tokens[cite: 8].
2. **Ventana de Contexto (El escritorio de trabajo):** Es la memoria a corto plazo de la IA durante una charla[cite: 8]. Si su escritorio es chico y lo llenas de papeles desordenados (miles de líneas de texto innecesario), se le caen hojas al piso: empieza a olvidar instrucciones iniciales o a inventar cosas (**alucinaciones**)[cite: 8].
3. **Temperatura (Nivel de creatividad):** Es un ajuste que controla qué tan predecible o arriesgada es la IA al elegir el siguiente token[cite: 8]. Temperatura baja = respuestas serias y repetibles; temperatura alta = respuestas más creativas pero más impredecibles.

---

2. Definiciones Clave del Nuevo Desarrollo de Software

¿Qué es el Desarrollo Agéntico?

Es una forma de crear software donde tú **no escribes línea por línea a mano**, sino que diriges a uno o varios **agentes de IA** (programas capaces de leer tu proyecto, crear archivos, ejecutar código y corregir errores por sí mismos) mientras tú supervisas el diseño y validas cada resultado[cite: 3, 12].

¿Qué es la Ingeniería Agéntica?

Es aplicar la disciplina de la ingeniería de software tradicional (arquitectura, pruebas, seguridad y orden) al trabajo con agentes[cite: 11, 12]. Es lo opuesto al *Vibe Coding* (pedir código por intuición y aceptarlo sin mirar)[cite: 11]. En la ingeniería agéntica, el humano diseña el sistema y garantiza que el software funcione a largo plazo[cite: 3, 11].

¿Qué es el Harness Engineering (Ingeniería de Arneses)?

Un modelo de IA solo (el LLM) es como el **motor de un auto** puesto arriba de una mesa: tiene mucha potencia, pero por sí solo no puede viajar por la ruta.

- **El Arnés (*Harness*):** Es todo el "auto" que construimos alrededor del motor (el volante, los pedales, el tablero y las herramientas) para que el LLM pueda conectarse a tu computadora, leer carpetas, usar la terminal y cumplir objetivos paso a paso[cite: 12, 15].
- **Los Guardarraíles (*Guardrails*):** Son las **barandas de seguridad de la ruta y los frenos del auto**. Son reglas y permisos que evitan que el agente rompa el proyecto: por ejemplo, prohibirle borrar bases de datos, obligarlo a pedir permiso antes de instalar librerías nuevas o exigirle que pase pruebas automáticas antes de dar una tarea por terminada[cite: 15, 16].
- **Context Engineering (Ingeniería de Contexto):** Es preparar **la mochila justa para la misión**. Consiste en seleccionar, organizar y actualizar únicamente la información exacta que el agente necesita en cada momento (ni de más para no saturar su ventana de contexto, ni de menos para que no adivine)[cite: 8, 15].

---

3. Herramientas que Permiten el Desarrollo Agéntico

Para dejar atrás el clásico "copiar y pegar" desde un chat web[cite: 12], hoy existen entornos diseñados específicamente como arneses agénticos[cite: 9, 13, 15]. Aunque cada uno tiene su propio enfoque, todos comparten la capacidad de leer tu carpeta de proyecto, ejecutar comandos y aplicar reglas de control[cite: 12, 13]:

- **OpenCode:** Es un agente de código abierto (*open-source*) muy popular que funciona desde la terminal, como aplicación de escritorio o dentro del editor[cite: 13]. Su mayor ventaja es ser **agnóstico al modelo** (puedes conectarle distintos proveedores de IA e incluso usar capas gratuitas) y contar con comandos nativos como `/init` para crear `AGENTS.md` o `Shift + TAB` para alternar entre los modos *Plan* y *Build*[cite: 13, 14, 19].
- **Claude Code:** Es la herramienta agéntica oficial de Anthropic[cite: 9, 13]. Funciona directamente en la terminal del sistema y destaca por su capacidad para analizar bases de código grandes, ejecutar pruebas, conectarse a servicios externos mediante **MCP** y trabajar en un ciclo estructurado de planificación antes de modificar archivos.

- **Codex (de OpenAI):** Es el entorno agéntico de OpenAI pensado para delegar tareas de ingeniería de forma autónoma[cite: 9]. Permite lanzar agentes en entornos aislados (*sandboxes*) para que escriban funcionalidades, corran tests, corrijan fallos y entreguen los cambios listos para que el desarrollador los revise y apruebe.
- **Google Antigravity:** Es el entorno de desarrollo agéntico de Google que integra editor, terminal y navegador web en un mismo espacio de trabajo[cite: 9]. Está diseñado para orquestar agentes que planifican tareas complejas, utilizan **Skills** y **MCP**, y generan artefactos verificables (como planes de implementación, capturas o pruebas en vivo) bajo supervisión humana.

> **Nota práctica para el aula:** Cambiar de herramienta no obliga a empezar de cero. Si mantienes bien documentados tus archivos `AGENTS.md` y `MEMORY.md` dentro de la carpeta del proyecto, cualquiera de estos agentes podrá leer las mismas reglas y continuar el trabajo con coherencia.

---

4. El Cambio de Paradigma: Del Teclado a la Orquestación

En 2026, el **90% de los desarrolladores** utiliza IA diariamente y el **45% del código nuevo** es generado por modelos, pero solo el **30%** es confiable sin supervisión humana[cite: 4].

- **Vibe Coding (Riesgo):** Describir intenciones y aceptar código sin auditar[cite: 11]. Genera deuda técnica, fallos de seguridad y sistemas inmantenibles[cite: 11].
- **Ingeniería Agéntica (Estándar profesional):** Operar en un nivel de abstracción superior validando especificaciones, arquitectura y resultados mediante arneses y guardarraíles[cite: 12, 15].

---

5. Cómo Acotar la Naturaleza Indeterminista de la IA

Como un LLM trabaja calculando probabilidades[cite: 7], si le pides lo mismo dos veces puede darte dos soluciones distintas. Para volver predecible el trabajo profesional:

- **Curaduría de contexto:** Entregar únicamente archivos, reglas y fragmentos relevantes (`@archivo`)[cite: 14, 15].
- **Gestión de sesiones:** Usar `/compact` para resumir sesiones largas o `/new` al cambiar de tarea y limpiar su escritorio de trabajo[cite: 14].
- **Selección de modelo por tarea:** Modelos de razonamiento profundo para pensar arquitectura; modelos medios para programar el día a día; modelos rápidos para tareas mecánicas[cite: 8, 9].
- **Validación determinista:** Usar herramientas exactas (compiladores, linters, tests unitarios y revisión manual de `/diff`) para comprobar que el código probabilístico funciona al 100%[cite: 14, 16].

---

6. Dónde es Estratégico Usar Markdown

Markdown es el formato ideal para comunicarse con agentes porque combina **estructura jerárquica clara** (títulos `#`, listas, bloques de código) con un **consumo mínimo de tokens** y total compatibilidad con Git[cite: 15].

Uso en el flujo	Propósito	Impacto en el Agente
Prompts estructurados	Separar `## Rol`, `## Contexto`, `## Tarea`, `## Restricciones` y `## Formato`[cite: 10, 14, 15].	Reduce la ambigüedad y evita que el modelo omita reglas críticas.
`AGENTS.md`	Constitución técnica y guardarraíles permanentes del repositorio[cite: 16].	El agente lo lee por defecto antes de actuar[cite: 16].
`MEMORY.md`	Bitácora viva con estado actual, decisiones y errores previos[cite: 18].	Evita repetir fallos entre sesiones distintas[cite: 18].
Especificaciones (SDD)	Definir requisitos y criterios de aceptación antes de programar[cite: 7].	Permite auditar la intención antes de generar código[cite: 12].
Comandos y Skills	Documentar flujos repetibles e instrucciones modulares[cite: 7].	Estandariza prácticas en todo el equipo.

7. Arquitectura del Contexto: `AGENTS.md` y `MEMORY.md`

`AGENTS.md` (Reglas permanentes · Máximo 30–40 líneas)[cite: 16]

Es el manual de instrucciones del proyecto[cite: 16]. Puede generarse una base inicial con `/init` y luego refinarse manualmente[cite: 14, 17]. Debe incluir:

- **Stack y estructura:** Tecnologías exactas y qué hace cada archivo clave[cite: 16].
- **Comandos:** Cómo ejecutar, compilar, pasar tests y linters[cite: 16].
- **Convenciones y trampas de dominio:** Reglas que no son obvias leyendo el código (ej. manejo de fechas locales vs. UTC)[cite: 16, 17].
- **Semáforo de límites (Guardarraíles):**
  - *Siempre:* Acciones obligatorias sin preguntar (ej. respetar idioma, actualizar memoria)[cite: 16, 18].
  - *Pregunta antes:* Crear archivos nuevos, instalar dependencias o alterar esquemas de datos[cite: 16].
  - *Nunca:* Prohibiciones absolutas (ej. no añadir frameworks, no tocar credenciales)[cite: 16, 17].

`MEMORY.md` (Estado evolutivo · Máximo ~50 líneas)[cite: 18]

Dado que la memoria de sesión del agente es efímera, este archivo conserva la continuidad entre ejecuciones[cite: 17, 18]:

- **Estado actual:** Funcionalidades operativas en la versión vigente[cite: 18].
- **Decisiones (y su porqué):** Razones arquitectónicas para que el agente no deshaga criterios previos[cite: 18].
- **Aprendizajes y errores a evitar:** Registro de fallos detectados[cite: 18].
- **Mantenimiento:** Si un aprendizaje se vuelve una regla fija, se mueve a `AGENTS.md`[cite: 18]. Nunca almacenar claves, tokens ni datos personales[cite: 18].

---

8. Extensibilidad del Arnés: Comandos, Skills y MCP

- **Comandos personalizados (`/comando`):** Atajos reutilizables que encapsulan prompts recurrentes (ej. revisión de seguridad o generación de tests)[cite: 7].
- **Agent Skills (`@skill`):** Paquetes de conocimiento e instrucciones especializadas que el agente carga bajo demanda para tareas concretas sin saturar el contexto global[cite: 7, 14].
- **MCP (*Model Context Protocol*):** Protocolo estándar (como un puerto USB universal para IA) que conecta al agente con herramientas externas, bases de datos, repositorios o APIs de forma controlada y segura[cite: 7].

---

9. Gobernanza de Ejecución: Modo Plan vs. Modo Build

En entornos agénticos (como OpenCode o Claude Code, alternando con `Shift + TAB`), separar la planificación de la ejecución es el principal guardarraíl operativo[cite: 14, 18, 19]:

- **Modo Plan (Análisis sin escritura):**
  - El agente explora el código, aclara dudas, identifica casos límite y propone un plan paso a paso sin modificar ningún archivo[cite: 18, 19].
  - *Cuándo usarlo:* Funcionalidades nuevas complejas, refactorizaciones grandes, decisiones de arquitectura o cambios en estructuras de datos[cite: 19].
- **Modo Build (Ejecución controlada):**
  - El agente modifica archivos, crea código y ejecuta comandos de terminal[cite: 12, 13, 14].
  - *Cuándo usarlo:* Correcciones puntuales obvias, prototipado rápido o inmediatamente después de haber aprobado un plan[cite: 19].

---

10. Metodología de Trabajo y Puntos de Control Obligatorios

Ciclo de Trabajo Recomendado

1. **Preparar el terreno:** Configurar `AGENTS.md` y redactar la especificación en Markdown[cite: 15, 16].
2. **Planificar (`Modo Plan`):** Pedir al agente el análisis de impacto, archivos a tocar y dudas antes de escribir código[cite: 18, 19].
3. **Ejecutar (`Modo Build`):** Implementar en iteraciones pequeñas y enfocadas[cite: 17].
4. **Verificar y persistir:** Ejecutar tests, revisar cambios con `/diff` (o revertir con `/undo` si se desvía) y actualizar `MEMORY.md`[cite: 14, 18].

Dónde debe existir Control Humano SIEMPRE

1. **Definición de requisitos y límites:** El agente no decide el alcance del producto ni las restricciones técnicas[cite: 15, 16].
2. **Aprobación del Plan:** Validar la estrategia antes de tocar archivos críticos[cite: 18].
3. **Esquemas de datos y dependencias:** Nunca permitir cambios silenciosos en bases de datos, `localStorage` o librerías externas[cite: 16, 17].
4. **Auditoría de `/diff` y seguridad:** Revisar cada cambio generado antes de hacer commit y asegurar que no se filtren secretos[cite: 14, 18].
5. **Curaduría de `AGENTS.md` y `MEMORY.md`:** Mantener ambos archivos breves, limpios y libres de contradicciones[cite: 16, 18].